AAI 520: Multi Agent Financial Analysis System

1. Project Overview

This notebook implements an autonomous Multi Agent Financial Analysis System built around a central **Investment Research Agent Supervisor**[cite: 2, 4]. Rather than relying on a single monolithic prompt, the system decomposes complex equity research queries into structured plans, dynamically routes tasks to domain specialists, synthesizes multi source evidence, critiques its own output via a quality gate, and retains enduring analytical takeaways across independent sessions.

Assignment Requirement & Rubric Mapping

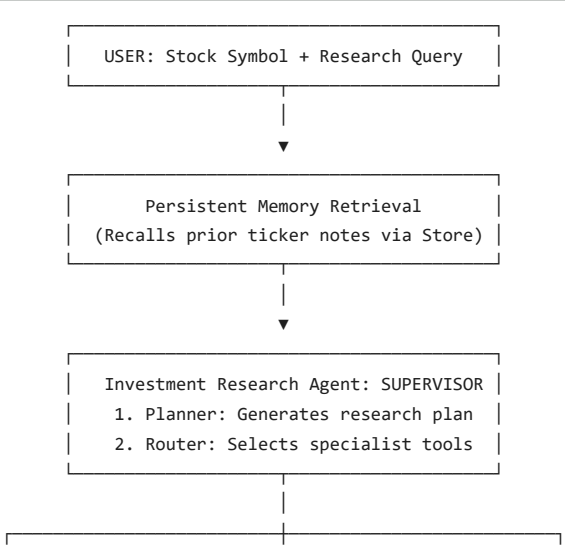
- **Plans research steps:** Implemented via `plan_research_steps()`, which generates an ordered investigation plan tailored to the user query and available specialist capabilities.
- **Dynamically uses tools:** Implemented via `get_market_data()`, `ingest_news()`, and `get_earnings_data()`, integrating live Yahoo Finance market/financial statement data and NewsAPI articles.
- **Self reflects:** Implemented via the **Evaluator Agent** (`evaluate_research_report()`), which audits drafts against specialist evidence across five quantitative criteria and flags critical factual issues.
- **Learns across runs:** Implemented via LangGraph's `InMemoryStore` and `SqliteSaver`, storing and recalling ticker specific analytical notes across sessions.
- **Workflow 1 (Prompt Chaining):** Implemented in the **News Agent** via a strict 5 stage sequential pipeline: Ingest → Preprocess → Classify → Extract → Summarize.
- **Workflow 2 (Routing):** Implemented via `select_research_tools()` and `AGENT_TOOL_REGISTRY`, dynamically dispatching queries only to relevant specialists (`market`, `news`, `earnings`) with explicit routing rationale.
- **Workflow 3 (Evaluator Optimizer):** Implemented via `run_evaluator_optimizer()` and `optimize_research_report()`, executing a closed loop generate → evaluate → refine cycle when quality thresholds are not met.

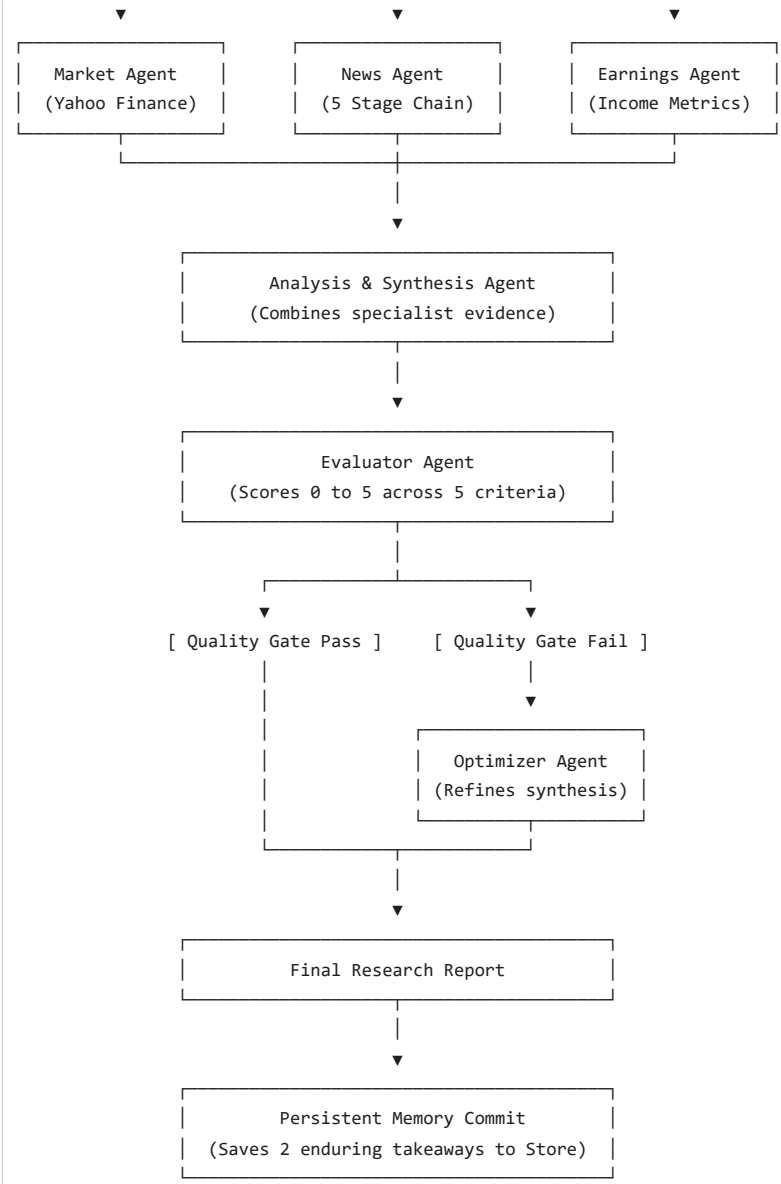
2. System Architecture

The system follows a hierarchical **Supervisor + Specialist Agents** design orchestrated within a LangGraph `StateGraph`:

1. **User Input:** Stock Ticker (`MSFT`) + Research Query.
2. **Memory Retrieval:** Recalls prior structural insights for the ticker from the persistent store.
3. **Investment Research Supervisor (Planner & Router):** Formulates step by step research goals and selects the required specialist subset.
4. **Specialist Execution Layer:**
    - **Market Agent:** Price action, historical returns, volatility, volume ratios, and valuation multiples.
    - **News Agent:** 5 stage prompt chain transforming raw headlines into executive catalysts.
    - **Earnings Agent:** Quarterly and annual income statement actuals and year over year growth rates.
5. **Analysis & Synthesis Agent:** Combines specialist evidence into a cohesive draft report.
6. **Evaluator & Optimizer Loop:** Scores the draft from 0 to 5 across five dimensions; if the quality gate fails, routes feedback to the Optimizer Agent for revision.
7. **Memory Persistence:** Extracts enduring takeaways from the verified report and commits them to long term storage.

System Execution Flow:





3. Technology Stack

- **Orchestration & State Management:** `LangGraph` (`StateGraph`, `SqliteSaver`, `InMemoryStore`)
- **Local LLM Inference:** `Ollama` running `llama3.1:8b` locally with deterministic JSON schema enforcement
- **Data Processing & Validation:** `Python 3`, `pandas`, `numpy`, `json`

4. Data Sources

- **Yahoo Finance (`yfinance`):** Provides 1 year daily OHLCV price history, company fundamentals, valuation ratios, and quarterly/annual reported income statements.
- **NewsAPI (`newsapi.org/v2/everything`):** Provides real time English financial news headlines and publisher metadata sorted by publication date.

Import Libraries

```
pip install langgraph langgraph-checkpoint-sqlite
```

```
Looking in indexes: https://packagefeedproxy.microsoft.io/pypi/simple/
Requirement already satisfied: langgraph in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.0
Requirement already satisfied: langgraph-checkpoint-sqlite in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\
Requirement already satisfied: langchain-core>=0.1 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pac
Requirement already satisfied: langgraph-checkpoint<5.0.0,>=2.1.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python3
Requirement already satisfied: langgraph-prebuilt<1.1.0,>=1.0.8 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314
Requirement already satisfied: langgraph-sdk<0.4.0,>=0.3.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\
Requirement already satisfied: pydantic>=2.7.4 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-package
Requirement already satisfied: xxhash>=3.5.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: ormsgpack>=1.12.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packa
Requirement already satisfied: httpx>=0.25.2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: orjson>=3.11.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
```

```
Requirement already satisfied: aiosqlite>=0.20 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-package
Requirement already satisfied: sqlite-vec>=0.1.6 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packa
Requirement already satisfied: anyio in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (from ht
Requirement already satisfied: certifi in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
Requirement already satisfied: httpcore==1.* in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: idna in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (from htt
Requirement already satisfied: h11>=0.16 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (frc
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: packaging>=23.2.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packa
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pa
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site
Requirement already satisfied: jsonpointer>=1.9 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packag
Requirement already satisfied: requests-toolbelt>=1.0.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: requests>=2.0.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-package
Requirement already satisfied: zstandard>=0.23.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packa
Requirement already satisfied: annotated-types>=0.6.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-
Requirement already satisfied: pydantic-core==2.41.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-p
Requirement already satisfied: typing-inspection>=0.4.2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: charset_normalizer<4,>=2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pack
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
pip install yfinance pandas numpy requests ollama python-dotenv
```

```
Looking in indexes: https://packagefeedproxy.microsoft.io/pypi/simple/
Requirement already satisfied: yfinance in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.7.
Requirement already satisfied: pandas in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.3.3)
Requirement already satisfied: numpy in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.4.2)
Requirement already satisfied: requests in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.32
Requirement already satisfied: ollama in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.6.3)
Requirement already satisfied: python-dotenv in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: beautifulsoup4>=4.11.1 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-
Requirement already satisfied: curl_cffi>=0.15 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-package
Requirement already satisfied: lxml>=4.9.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (f
Requirement already satisfied: multitasking>=0.0.7 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pac
Requirement already satisfied: peewee>=3.16.2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: platformdirs>=2.0.0 in C:\Users\kugurtoo\AppData\Roaming\Python\Python314\site-packages (from
Requirement already satisfied: protobuf>=3.19.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packag
Requirement already satisfied: pytz>=2022.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (
Requirement already satisfied: websockets>=13.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packag
Requirement already satisfied: python-dateutil>=2.8.2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-
Requirement already satisfied: tzdata>=2022.7 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: charset_normalizer<4,>=2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: idna<4,>=2.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pack
Requirement already satisfied: certifi>=2017.4.17 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-pack
Requirement already satisfied: httpx>=0.27 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (f
Requirement already satisfied: pydantic>=2.9 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: soupsieve>=1.6.1 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packag
Requirement already satisfied: typing-extensions>=4.0.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: cffi>=2.0.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (f
Requirement already satisfied: pycparser in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (frc
Requirement already satisfied: anyio in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (from ht
Requirement already satisfied: httpcore==1.* in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requirement already satisfied: h11>=0.16 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (frc
Requirement already satisfied: annotated-types>=0.6.0 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-
Requirement already satisfied: pydantic-core==2.41.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-p
Requirement already satisfied: typing-inspection>=0.4.2 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\sit
Requirement already satisfied: six>=1.5 in c:\Users\kugurtoo\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
#%pip install yfinance ollama
import yfinance as yf
import pandas as pd
import numpy as np
import ollama
import json
```

Variables

```
import os

TICKER_SYMBOL = 'MSFT'  # Shared ticker used by all agent examples
OLLAMA_MODEL = 'llama3.1:8b'  # Shared model used by all agents
```

```
os.environ["NEWS_API_KEY"] = "38f58eb3a57444e28cd041f14d466a87"
```

Utility Functions

```
from getpass import getpass

def ensure_news_api_key() -> str:
    api_key = os.environ['NEWS_API_KEY'] or get_secret('NEWS_API_KEY')
    if not api_key:
        api_key = getpass('NewsAPI key (input hidden): ').strip()
        if not api_key:
            raise RuntimeError('A NewsAPI key is required because the News specialist was selected.')
        os.environ['NEWS_API_KEY'] = api_key
    return api_key
```

LLM Setup

```
%%bash
apt-get update -qq
apt-get install -y -qq zstd

# 1. Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 2. Start the server in the background
nohup ollama serve > ollama.log 2>&1 &
sleep 5

# 4. Download the model
echo "Pulling the Llama model..."
ollama pull $OLLAMA_MODEL
echo "Setup Complete!"
```

```
Couldn't find program: 'bash'
```

Local LLM (If Installed)

```
import requests

response = requests.get("http://localhost:11434")
print(response.text)  # Should output: "Ollama is running"
```

```
Ollama is running
```

```
import ollama

try:
    # Fetch list of downloaded models
    models_response = ollama.list()
    print("🟢 Ollama server is ONLINE.")
    print("Downloaded models available:")
    for model in models_response.get("models", []):
        print(f" - {model['model']}")
except Exception as e:
    print("🔴 Ollama server is OFFLINE or unreachable.")
    print(f"Error details: {e}")
```

```
🟢 Ollama server is ONLINE.
Downloaded models available:
 - llama3.1:8b
```

5. Specialist Agent Design & Workflow 1: Prompt Chaining

5.1 Market Agent (`get_market_data` & `market_agent`)

The Market Agent retrieves factual market data via `yfinance` and computes multi horizon returns (1M, 3M, 6M), annualized volatility ($\sigma \times \sqrt{252}$), 52 week high/low distance, trading volume ratios, and valuation metrics (trailing P/E, forward P/E, EPS, market cap)[cite: 7, 26]. The LLM is strictly constrained to interpret only the supplied JSON metrics without inventing figures or offering personalized investment advice.

```
def get_market_data(ticker_symbol: str) -> dict:
    """
    Retrieve structured market information for a stock using Yahoo Finance.

    Parameters
    ----------
    ticker_symbol : str
        Stock ticker symbol, e.g. AAPL, MSFT, NVDA.

    Returns
    -------
    dict
        Structured market information including:
        - Current price and daily performance
        - 1M, 3M, and 6M returns
        - Annualized volatility
        - 52-week high/low and position relative to them
        - Trading volume and volume ratio
        - Market capitalization
        - Valuation metrics
        - Revenue, net income, and EPS
        - Data timestamp
    """

    # ------------------------------
    # Clean ticker symbol
    # ------------------------------
    ticker_symbol = ticker_symbol.upper().strip()

    # ------------------------------
    # Create Yahoo Finance ticker
    # ------------------------------
    ticker = yf.Ticker(ticker_symbol)

    # ------------------------------
    # Company information
    # ------------------------------
    info = ticker.info

    # ------------------------------
    # Historical price data
    # ------------------------------
    history = ticker.history(period="1y")

    if history.empty:
        raise ValueError(
            f"No market data found for ticker: {ticker_symbol}"
        )

    if len(history) < 2:
        raise ValueError(
            f"Insufficient historical data for ticker: {ticker_symbol}"
        )

    # ------------------------------
    # Current market information
    # ------------------------------
    current_price = history["Close"].iloc[-1]
    previous_close = history["Close"].iloc[-2]

    daily_change = current_price - previous_close

    daily_change_pct = (
        daily_change / previous_close
    ) * 100

    # ------------------------------
    # Returns
    # Approximately:
    # 21 trading days  = 1 month
    # 63 trading days  = 3 months
    # 126 trading days = 6 months
    # ------------------------------
    def calculate_return(days):
        if len(history) > days:
            old_price = history["Close"].iloc[-days]
            return float(
                ((current_price - old_price) / old_price) * 100
            )

        return None
```

```
        return_1m = calculate_return(21)
        return_3m = calculate_return(63)
        return_6m = calculate_return(126)

        # ------------------------------
        # Volatility
        # ------------------------------
        daily_returns = history["Close"].pct_change().dropna()

        annualized_volatility = (
            daily_returns.std() * np.sqrt(252) * 100
        )

        # ------------------------------
        # 52-week statistics
        # ------------------------------
        week_52_high = history["High"].max()
        week_52_low = history["Low"].min()

        # Distance from 52-week high
        distance_from_52w_high_pct = (
            (current_price - week_52_high)
            / week_52_high
        ) * 100

        # Distance from 52-week low
        distance_from_52w_low_pct = (
            (current_price - week_52_low)
            / week_52_low
        ) * 100

        # ------------------------------
        # Volume
        # ------------------------------
        average_volume = history["Volume"].mean()
        latest_volume = history["Volume"].iloc[-1]

        # Volume relative to average
        volume_ratio = (
            latest_volume / average_volume
            if average_volume > 0
            else None
        )

        # ------------------------------
        # Company fundamentals
        # ------------------------------
        market_cap = info.get("marketCap")
        pe_ratio = info.get("trailingPE")
        forward_pe = info.get("forwardPE")
        revenue = info.get("totalRevenue")
        net_income = info.get("netIncomeToCommon")

        # EPS
        eps = info.get("trailingEps")

        # ------------------------------
        # Data timestamp
        # ------------------------------
        data_timestamp = history.index[-1].isoformat()

        # ------------------------------
        # Return structured data
        # ------------------------------
        return {
            # Identification
            "ticker": ticker_symbol,
            "company_name": info.get("longName"),

            # Data timestamp
            "data_timestamp": data_timestamp,

            # Current market position
            "current_price": float(current_price),
            "previous_close": float(previous_close),

            # Daily performance
            "daily_change": float(daily_change),
            "daily_change_pct": float(daily_change_pct),

            # Historical performance
            "return_1m_pct": return_1m,
```

```
        "return_3m_pct": return_3m,
        "return_6m_pct": return_6m,

        # Volatility
        "annualized_volatility_pct": float(
            annualized_volatility
        ),

        # 52-week statistics
        "52_week_high": float(week_52_high),
        "52_week_low": float(week_52_low),

        "distance_from_52w_high_pct": float(
            distance_from_52w_high_pct
        ),

        "distance_from_52w_low_pct": float(
            distance_from_52w_low_pct
        ),

        # Trading activity
        "latest_volume": int(latest_volume),
        "average_volume": int(average_volume),
        "volume_ratio": float(volume_ratio)
        if volume_ratio is not None
        else None,

        # Valuation
        "market_cap": market_cap,
        "pe_ratio": pe_ratio,
        "forward_pe": forward_pe,

        # Financial performance
        "revenue": revenue,
        "net_income": net_income,
        "eps": eps,
    }
```

Sample output of function to be called by Market Agent

```
data = get_market_data(TICKER_SYMBOL)

for key, value in data.items():
    print(f"{key}: {value}")
```

```
ticker: MSFT
company_name: Microsoft Corporation
data_timestamp: 2026-10-09T00:00:00-04:00
current_price: 535.0700073242188
previous_close: 522.6099853515625
daily_change: 12.46002197265625
daily_change_pct: 2.3841913323325286
return_1m_pct: 7.957549392259152
return_3m_pct: 39.26617391410715
return_6m_pct: 39.771150031795564
annualized_volatility_pct: 32.818217943648946
52_week_high: 549.2013167264516
52_week_low: 348.54386902384044
distance_from_52w_high_pct: -2.573065462854938
distance_from_52w_low_pct: 53.515828243594754
latest_volume: 18483600
average_volume: 31800333
volume_ratio: 0.5812391806555741
market_cap: 3973186846720
pe_ratio: 29.111536
forward_pe: 22.59842
revenue: 331839012864
net_income: 133748998144
eps: 18.38
```

Market Agent

```
def market_agent(ticker_symbol: str) -> str:
    """
    Analyze market data for a given stock ticker.

    Parameters
    ----------
    ticker_symbol : str
```

```
            Stock ticker symbol such as AAPL, MSFT, or NVDA.

        Returns
        -------
        str
            LLM-generated market analysis.
        """

        # ---------------------------------
        # Step 1: Retrieve market data
        # ---------------------------------

        market_data = get_market_data(ticker_symbol)

        # ---------------------------------
        # Step 2: Convert data to JSON
        # ---------------------------------

        market_data_json = json.dumps(
            market_data,
            indent=2,
            default=str
        )

        # ---------------------------------
        # Step 3: Create LLM prompt
        # ---------------------------------

        prompt = f"""
    You are a financial market analysis specialist.

    Analyze the following factual market data.

    IMPORTANT:
    - Use only the supplied data.
    - Do not invent prices or financial metrics.
    - Clearly distinguish facts from interpretation.
    - Do not provide personalized investment advice.
    - Identify important trends and risks.
    - If a value is missing, say that it is unavailable.

    Market data:

    {market_data_json}

    Provide your analysis using this structure:

    1. Current Market Position
    2. Recent Performance
    3. Volatility
    4. Trading Activity
    5. Valuation Metrics
    6. Key Observations
    7. Risks and Limitations
    """

        # ---------------------------------
        # Step 4: Ask Llama
        # ---------------------------------

        response = ollama.chat(
            model=OLLAMA_MODEL,
            messages=[
                {
                    "role": "system",
                    "content": (
                        "You are a financial market "
                        "analysis assistant."
                    )
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ]
        )

        # ---------------------------------
        # Step 5: Return result
        # ---------------------------------

        return response["message"]["content"]
```

Sample output from Market Agent

```
result = market_agent(TICKER_SYMBOL)
print(result)
```

```
**Market Analysis Report for MSFT (Microsoft Corporation)**

**1. Current Market Position**

* Current Price: $535.07
* Previous Close: $522.61
* Daily Change: $12.46 (2.38% increase)
* Market Capitalization: $3,973,186,847,420

**2. Recent Performance**

* 1-Month Return: 7.96%
* 3-Month Return: 39.27%
* 6-Month Return: 39.77%
* Annualized Volatility: 32.82%

**3. Volatility**

* Annualized Volatility: 32.82% (high volatility)
* Distance from 52-Week High: -2.57% (below recent highs)
* Distance from 52-Week Low: 53.52% (well above recent lows)

**4. Trading Activity**

* Latest Volume: 18,483,600
* Average Volume: 31,800,333
* Volume Ratio: 0.58 (below average volume)

**5. Valuation Metrics**

* Price-to-Earnings (P/E) Ratio: 29.11
* Forward P/E Ratio: 22.60
* Price-to-Earnings (P/E) Ratio compared to industry average: unavailable

**6. Key Observations**

* MSFT has experienced significant price appreciation over the past 3 and 6 months, with returns of 39.27% and 39.77%, respe
* The stock is trading at a relatively high P/E ratio of 29.11, which may indicate a high valuation.
* The forward P/E ratio is lower than the current P/E ratio, suggesting a potential decrease in future earnings expectations
* MSFT has high annualized volatility, indicating significant price swings.

**7. Risks and Limitations**

* The high P/E ratio and volatility may indicate a high-risk investment.
* The company's future earnings and revenue growth are uncertain and may impact the stock price.
* The data does not provide information on industry trends, competitive landscape, or company-specific events that may impac
* The analysis is based solely on the provided market data and does not consider any external factors that may influence the
```

```
import json
import ollama
import requests
try:
    from google.colab import userdata
except ImportError:
    userdata = None


def get_secret(name: str, default=None):
    if userdata is None:
        return default
    try:
        return userdata.get(name)
    except Exception:
        return default
```

5.3 News Agent (`news_agent`)

The News Agent fetches real time financial news articles from NewsAPI and processes them through a five stage sequential pipeline (Ingestion, Preprocessing, Classification, Information Extraction, and Executive Summarization).

```
import os
import requests
import ollama


# STAGE 1: Ingestion
```

```
def ingest_news(ticker_symbol: str) -> list[dict[str, str]]:
    api_key = os.environ.get('NEWS_API_KEY') or get_secret('NEWS_API_KEY')
    if not api_key:
        raise RuntimeError('Set NEWS_API_KEY in the environment or Colab Secrets before fetching news.')
    response = requests.get(
        'https://newsapi.org/v2/everything',
        params={'q': ticker_symbol, 'sortBy': 'publishedAt', 'language': 'en'},
        headers={'X-Api-Key': api_key},
        timeout=20,
    )
    response.raise_for_status()
    data = response.json()
    if data.get('status') != 'ok':
        raise RuntimeError(f"NewsAPI request failed: {data.get('message', 'Unknown API error')}")
    articles = []
    for item in data.get('articles', [])[:5]:
        source = item.get('source') or {}
        articles.append({'title': item.get('title') or 'No Title', 'publisher': source.get('name') or 'Unknown'})
    if not articles:
        raise ValueError(f'No real news found for {ticker_symbol}. Check the ticker and API key.')
    return articles


# STAGE 2: Preprocessing
def preprocess_news(articles: list[dict[str, str]]) -> str:
    return '\n'.join(
        f"Publisher: {article.get('publisher', '').strip()} | Headline: {article.get('title', '').strip()}"
        for article in articles
    )


def _ask_ollama(prompt: str) -> str:
    response = ollama.chat(model=OLLAMA_MODEL, messages=[{'role': 'user', 'content': prompt}])
    return response['message']['content']


# STAGE 3: Classification
def classify_news(cleaned_text: str) -> str:
    prompt = f'''You are a financial news classification specialist.
Analyze these headlines:
{cleaned_text}
Classify each story as Earnings, Market Trends, Product Updates, or Macroeconomics. Determine overall sentiment as Bullish,
    return _ask_ollama(prompt)


# STAGE 4: Extraction
def extract_information(cleaned_text: str, classified_text: str) -> str:
    prompt = f'''You are a financial data extraction specialist. Review the original headlines and classifications.
Original Headlines:
{cleaned_text}
Classifications:
{classified_text}
Extract concrete metrics, dates, and events. Include full event context and output a bulleted list of complete factual stat
    return _ask_ollama(prompt)


# STAGE 5: Summarization
def summarize_news(extracted_facts: str) -> str:
    prompt = f'''You are an executive investment research analyst. Using only these extracted facts, produce a concise summ
{extracted_facts}
Use exactly these headers:
1. Executive Overview
2. Key Catalysts and Drivers
3. Immediate Market Implications
Output only the summary, without conversational filler.'''
    return _ask_ollama(prompt)


def news_agent(ticker_symbol: str) -> dict[str, str]:
    print(f'Fetching real-world news for {ticker_symbol}...')
    cleaned_news = preprocess_news(ingest_news(ticker_symbol))
    print('New Agent is classifying news...')
    classified_data = classify_news(cleaned_news)
    print('New Agent is extracting facts...')
    extracted_facts = extract_information(cleaned_news, classified_data)
    print('New Agent is writing final summary...')
    final_summary = summarize_news(extracted_facts)
    return {
        'preprocessed_input': cleaned_news,
        'classification_output': classified_data,
        'extracted_facts_output': extracted_facts,
        'summary_output': final_summary,
```

```
    }


_previous_news_api_key = os.environ.get('NEWS_API_KEY')
try:
    news_results = news_agent(TICKER_SYMBOL)
finally:
    if _previous_news_api_key is None:
        os.environ.pop('NEWS_API_KEY', None)
    else:
        os.environ['NEWS_API_KEY'] = _previous_news_api_key

print('\n=== STAGE 2: PREPROCESSED TEXT ===')
print(news_results['preprocessed_input'])
print('\n=== STAGE 3: CLASSIFICATION ===')
print(news_results['classification_output'])
print('\n=== STAGE 4: EXTRACTION ===')
print(news_results['extracted_facts_output'])
print('\n=== STAGE 5: FINAL SUMMARY ===')
print(news_results['summary_output'])
```

```
Fetching real-world news for MSFT...
New Agent is classifying news...
New Agent is extracting facts...
New Agent is writing final summary...

=== STAGE 2: PREPROCESSED TEXT ===
Publisher: Pypi.org | Headline: finzdata added to PyPI
Publisher: Biztoc.com | Headline: S&P 500, Nasdaq 100, Dow End Week Higher With Traders Eyeing Earnings Season — MSFT, NFL
Publisher: Biztoc.com | Headline: Are You Overpaying For Microsoft Stock Versus Its Rivals?
Publisher: The Times of India | Headline: Microsoft stock (MSFT) falls as US targets green card filings for H-1B workers a
Publisher: PRNewswire | Headline: $2.6 Trillion and Climbing: Where the AI Spending Is Going

=== STAGE 3: CLASSIFICATION ===
**Publisher: Pypi.org | Headline: finzdata added to PyPI**
* Classification: Product Updates
* Sentiment: Neutral

**Publisher: Biztoc.com | Headline: S&P 500, Nasdaq 100, Dow End Week Higher With Traders Eyeing Earnings Season — MSFT, N
* Classification: Market Trends
* Sentiment: Bullish

**Publisher: Biztoc.com | Headline: Are You Overpaying For Microsoft Stock Versus Its Rivals?**
* Classification: Market Trends
* Sentiment: Bearish

**Publisher: The Times of India | Headline: Microsoft stock (MSFT) falls as US targets green card filings for H-1B workers
* Classification: Macroeconomics
* Sentiment: Bearish

**Publisher: PRNewswire | Headline: $2.6 Trillion and Climbing: Where the AI Spending Is Going**
* Classification: Macroeconomics
* Sentiment: Bullish

=== STAGE 4: EXTRACTION ===
After reviewing the original headlines and classifications, I've extracted the following concrete metrics, dates, and even

* **Financial metrics:**
        + $2.6 trillion: The current AI spending, which is "Climbing" according to the PRNewswire headline.
        + Trillion-dollar investment in AI: The headline implies a significant investment in AI, but does not specify the
* **Stock performance:**
        + Microsoft stock (MSFT) fell: According to The Times of India headline, the stock fell due to US targets green ca
        + Microsoft stock (MSFT) is in focus: According to the Biztoc.com headline, the stock is among those being eyed by
* **Market indices:**
        + S&P 500, Nasdaq 100, and Dow ended the week higher: According to the Biztoc.com headline, these market indices c
* **Earnings season:**
        + Earnings season is in focus: According to the Biztoc.com headline, traders are eyeing earnings season for compan
        + Specific companies being eyed:
                - Microsoft (MSFT)
                - Netflix (NFLX)
                - SPCX (no further information available)
                - ASTS (no further information available)
* **Regulatory events:**
        + US targets green card filings for H-1B workers: According to The Times of India headline, the US has targeted gr
        + Italy probes gaming business: According to The Times of India headline, Italy is investigating a gaming business
* **Product updates:**
        + finzdata added to PyPI: According to the Pypi.org headline, the finzdata package has been added to the Python Pa
```

5.4 Earnings Agent (`get_earnings_data` & `earnings_agent`)

The Earnings Agent extracts up to 5 periods of quarterly and annual reported income statements from Yahoo Finance (`Total Revenue`, `Diluted/Basic EPS`, and `Net Income`), calculates exact year over year percentage changes, and explicitly flags unavailable management guidance so the model never hallucinates forward projections.

```
import json
import ollama
import pandas as pd
import yfinance as yf


def _find_statement_row(statement, candidates):
    available = {str(label).strip().lower(): label for label in statement.index}
    for candidate in candidates:
        label = available.get(candidate.lower())
        if label is not None:
            return statement.loc[label]
    return None


def _earnings_periods(statement, limit):
    if statement.empty:
        return []

    periods = sorted(statement.columns, key=lambda period: pd.Timestamp(period), reverse=True)
    metric_rows = {
        'revenue': _find_statement_row(statement, ['Total Revenue', 'Operating Revenue']),
        'eps': _find_statement_row(statement, ['Diluted EPS', 'Basic EPS']),
        'net_income': _find_statement_row(
            statement,
            ['Net Income Common Stockholders', 'Net Income', 'Net Income Including Noncontrolling Interests'],
        ),
    }
    records = []
    for period in periods[:limit]:
        metrics = {}
        for metric, row in metric_rows.items():
            value = None if row is None else row.get(period)
            metrics[metric] = None if value is None or pd.isna(value) else float(value)
        records.append({'period_end': str(pd.Timestamp(period).date()), **metrics})
    return records


def _year_over_year_change(current, previous):
    if current is None or previous is None or previous == 0:
        return None
    return round(((current - previous) / abs(previous)) * 100, 2)


def get_earnings_data(ticker_symbol: str) -> dict:
    """Get Yahoo Finance reported income statements and year-over-year changes."""
    ticker_symbol = ticker_symbol.upper().strip()
    if not ticker_symbol:
        raise ValueError('Ticker symbol cannot be empty.')

    ticker = yf.Ticker(ticker_symbol)
    quarterly = _earnings_periods(ticker.quarterly_income_stmt, 5)
    annual = _earnings_periods(ticker.income_stmt, 5)
    if not quarterly and not annual:
        raise ValueError(f'No Yahoo Finance income statement data found for {ticker_symbol}.')

    quarterly_yoy = {}
    if len(quarterly) >= 5:
        for metric in ('revenue', 'eps', 'net_income'):
            quarterly_yoy[metric] = _year_over_year_change(
                quarterly[0][metric], quarterly[4][metric]
            )

    annual_yoy = {}
    if len(annual) >= 2:
        for metric in ('revenue', 'eps', 'net_income'):
            annual_yoy[metric] = _year_over_year_change(
                annual[0][metric], annual[1][metric]
            )

    return {
        'ticker': ticker_symbol,
        'source': 'Yahoo Finance reported income statements',
        'quarterly_actuals': quarterly,
        'annual_actuals': annual,
        'year_over_year_change_pct': {
            'latest_quarter_vs_same_quarter_last_year': quarterly_yoy,
            'latest_fiscal_year_vs_prior_fiscal_year': annual_yoy,
        },
        'management_guidance': (
            'Not available in Yahoo Finance income statements. Consult the latest company earnings release or SEC filing.'
```

```
        ),
    }


def earnings_agent(ticker_symbol: str) -> str:
    """Analyze reported earnings and year-over-year changes with the local Ollama model."""
    earnings_data = get_earnings_data(ticker_symbol)
    earnings_data_json = json.dumps(earnings_data, indent=2, default=str)

    prompt = f'''You are a financial earnings analysis specialist.
Analyze only the factual Yahoo Finance data below.

Rules:
- Do not invent or infer missing financial values or management guidance.
- Distinguish reported actuals from interpretation.
- Treat null year-over-year values as unavailable.
- This is informational analysis, not personalized investment advice.

Earnings data:
{earnings_data_json}

Use these headings:
1. Revenue
2. EPS
3. Net Income
4. Guidance
5. Year-over-Year Changes
6. Key Observations and Limitations'''

    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[
            {'role': 'system', 'content': 'You are a careful financial earnings analysis assistant.'},
            {'role': 'user', 'content': prompt},
        ],
    )
    return response['message']['content']


earnings_result = earnings_agent(TICKER_SYMBOL)
print(earnings_result)
```

```
**1. Revenue**

* Latest quarter (2026-06-30): $90,007,000,000
* Latest year (2026-06-30): $331,839,000,000
* Prior year (2025-06-30): $281,724,000,000
* Second prior year (2024-06-30): $245,122,000,000
* Third prior year (2023-06-30): $211,915,000,000

**2. EPS**

* Latest quarter (2026-06-30): $4.81
* Latest year (2026-06-30): $17.95
* Prior year (2025-06-30): $13.64
* Second prior year (2024-06-30): $11.80
* Third prior year (2023-06-30): $9.68

**3. Net Income**

* Latest quarter (2026-06-30): $35,766,000,000
* Latest year (2026-06-30): $133,749,000,000
* Prior year (2025-06-30): $101,832,000,000
* Second prior year (2024-06-30): $88,136,000,000
* Third prior year (2023-06-30): $72,361,000,000

**4. Guidance**

* Not available in Yahoo Finance income statements. Consult the latest company earnings release or SEC filing.

**5. Year-over-Year Changes**

* Latest quarter vs same quarter last year:
        + Revenue: 17.75%
        + EPS: 31.78%
        + Net Income: 31.33%
* Latest year vs prior year:
        + Revenue: 17.79%
        + EPS: 31.6%
        + Net Income: 31.34%

**6. Key Observations and Limitations**

* The data is based on Yahoo Finance reported income statements and may not reflect the company's official financials.
* Year-over-year changes for the first quarter (2026-03-31) are not available as there is no prior year data.
* The latest fiscal year (2026-06-30) data is only available for the latest quarter, and the prior year (2025-06-30) data is
```

6. Autonomous Agent Functions, Workflow 2 (Routing), & Workflow 3 (Evaluator Optimizer)

6.1 Planning (`plan_research_steps`) & 6.2 Dynamic Tool Selection (`select_research_tools`)

- **Research Planning:** Given a ticker and user research question, the planner inspects `AGENT_TOOL_REGISTRY` descriptions and outputs an ordered JSON array of investigation steps.
- **Workflow 2 (Routing):** Rather than blindly executing every tool, the router analyzes the scope of the user question, selects only the necessary registered specialists (`market`, `news`, `earnings`), validates that every chosen tool exists in the registry, and provides a clear `routing_rationale`.

6.3 Self Reflection & Section 9: Workflow 3 (Evaluator Optimizer)

To guarantee analytical rigor, the draft synthesis passes through an independent quality gate:

- **Evaluator Agent (`evaluate_research_report`):** Audits the draft against raw specialist evidence, scoring five dimensions from 0 to 5 (`factual_grounding`, `completeness`, `source_attribution`, `financial_reasoning`, `clarity`) and compiling `critical_issues`, `issues`, and `revision_instructions`.
- **Quality Gate (`passes_quality_gate`):** Requires an arithmetic mean score ≥ 4.0, zero critical issues, and every individual criterion score ≥ 3.0[cite: 26].
- **Optimizer Agent (`optimize_research_report`):** If the initial draft fails the quality gate, the Optimizer revises the report using the Evaluator's feedback, re evaluates the revised draft, and selects the higher ranking report (penalizing critical issues by $1.5\times$).

```
AGENT_TOOL_REGISTRY = {
    'market': {
        'description': 'Yahoo Finance prices, returns, volatility, volume, and valuation metrics.',
        'run': market_agent,
    },
    'news': {
        'description': 'NewsAPI headlines processed through ingestion, preprocessing, classification, extraction, and summa
        'run': news_agent,
    },
    'earnings': {
        'description': 'Yahoo Finance reported revenue, EPS, net income, and year-over-year comparisons.',
        'run': earnings_agent,
    },
}
```

functions

```
def _agent_json(prompt: str) -> dict:
    response = ollama.chat(
        model=OLLAMA_MODEL,
        format='json',
        messages=[{'role': 'user', 'content': prompt}],
    )
    result = json.loads(response['message']['content'])
    if not isinstance(result, dict):
        raise ValueError('The model must return a JSON object.')
    return result

def _agent_text(prompt: str) -> str:
    response = ollama.chat(
        model=OLLAMA_MODEL,
        messages=[{'role': 'user', 'content': prompt}],
    )
    return response['message']['content']
```

5.5 Evaluator & Optimizer Agents (`evaluate_research_report` & `optimize_research_report`)

The Evaluator Agent independently audits draft research reports against raw specialist evidence across five quantitative criteria (`factual_grounding`, `completeness`, `source_attribution`, `financial_reasoning`, `clarity`) from 0 to 5 and flags critical factual errors. When a draft fails the quality gate, the Optimizer Agent revises the report using the Evaluator's specific revision instructions.

```
import json


def evaluate_research_report(
```

```
    ticker_symbol: str,
    research_question: str,
    specialist_evidence: dict,
    draft_report: str,
) -> dict:
    """
    Evaluate a financial research report against its evidence.

    Returns structured scores, issues, and revision instructions.
    """

    evidence_json = json.dumps(
        specialist_evidence,
        indent=2,
        default=str,
    )

    prompt = f"""
You are an independent financial research quality evaluator.

Evaluate the draft report using ONLY the supplied evidence.
Do not assume that a claim is true merely because it appears
in the draft.

TICKER:
{ticker_symbol}

USER RESEARCH QUESTION:
{research_question}

SPECIALIST EVIDENCE:
{evidence_json}

DRAFT REPORT:
{draft_report}

Score each criterion from 0 to 5:

1. factual_grounding:
   Are factual claims supported by the evidence?

2. completeness:
   Does the report answer the user's question?

3. source_attribution:
   Are sources and evidence limitations identified?

4. financial_reasoning:
   Are financial metrics interpreted correctly?

5. clarity:
   Is the report clear, structured, and appropriately qualified?

Also identify:
- critical_issues: material factual errors, invented figures,
  contradictions, or unsupported consequential claims.
- issues: other weaknesses that should be corrected.
- revision_instructions: specific actions for improving the draft.

A critical issue must not be ignored because other scores are high.

Return ONLY a JSON object with this schema:
{{
    "scores": {{
        "factual_grounding": 0,
        "completeness": 0,
        "source_attribution": 0,
        "financial_reasoning": 0,
        "clarity": 0
    }},
    "critical_issues": [],
    "issues": [],
    "revision_instructions": [],
    "overall_score": 0.0
}}

overall_score must be the arithmetic mean of the five scores.
"""

    result = _agent_json(prompt)

    required_scores = [
```

```
            "factual_grounding",
            "completeness",
            "source_attribution",
            "financial_reasoning",
            "clarity",
        ]

        scores = result.get("scores")

        if not isinstance(scores, dict):
            raise ValueError("Evaluator returned invalid scores.")

        # Validate every score instead of trusting the LLM output.
        validated_scores = {}

        for criterion in required_scores:
            score = scores.get(criterion)

            if (
                isinstance(score, bool)
                or not isinstance(score, (int, float))
                or not 0 <= score <= 5
            ):
                raise ValueError(
                    f"Invalid evaluator score: {criterion}"
                )

            validated_scores[criterion] = float(score)

        for field in [
            "critical_issues",
            "issues",
            "revision_instructions",
        ]:
            if (
                not isinstance(result.get(field), list)
                or not all(
                    isinstance(item, str)
                    for item in result[field]
                )
            ):
                raise ValueError(
                    f"Evaluator returned invalid field: {field}"
                )

        # Calculate the score ourselves rather than trusting
        # the LLM's overall_score.
        overall_score = round(
            sum(validated_scores.values())
            / len(validated_scores),
            2,
        )

        return {
            "scores": validated_scores,
            "overall_score": overall_score,
            "critical_issues": result["critical_issues"],
            "issues": result["issues"],
            "revision_instructions": result[
                "revision_instructions"
            ],
        }
```

Optimizer Agent

```
def optimize_research_report(
    ticker_symbol: str,
    research_question: str,
    specialist_evidence: dict,
    draft_report: str,
    evaluation: dict,
) -> str:
    """
    Improve a research report using evaluator feedback.
    """

    evidence_json = json.dumps(
        specialist_evidence,
        indent=2,
        default=str,
    )
```

```
    evaluation_json = json.dumps(
        evaluation,
        indent=2,
    )

    prompt = f"""
You are a financial research report optimizer.

Revise the draft using the evaluator's feedback.

TICKER:
{ticker_symbol}

USER QUESTION:
{research_question}

ORIGINAL SPECIALIST EVIDENCE:
{evidence_json}

CURRENT DRAFT:
{draft_report}

EVALUATOR FEEDBACK:
{evaluation_json}

Instructions:

1. Correct factual errors identified by the evaluator.
2. Address every material critical issue.
3. Improve completeness and financial reasoning.
4. Improve source attribution and clarity.
5. Preserve claims that are already supported.
6. Do not invent financial figures, events, or sources.
7. If evidence is missing, explicitly state the limitation.
8. Do not claim a correction is verified unless the evidence
   supports it.
9. Do not provide personalized investment advice.

Return ONLY the complete revised report.
Do not return commentary about the revision process.
"""

    revised_report = _agent_text(prompt).strip()

    if not revised_report:
        raise ValueError("Optimizer returned an empty report.")

    return revised_report
```

Evaluator-Optimizer Controller

```
def run_evaluator_optimizer(
    ticker_symbol: str,
    research_question: str,
    specialist_evidence: dict,
    initial_report: str,
    minimum_score: float = 4.0,
) -> dict:
    """
    Evaluate a draft, optionally refine it once, and select
    the best candidate based on evaluator feedback.

    Maximum: two evaluations and one optimization.
    """

    current_report = initial_report

    initial_evaluation = evaluate_research_report(
        ticker_symbol,
        research_question,
        specialist_evidence,
        current_report,
    )

    evaluations = [initial_evaluation]

    def passes_quality_gate(evaluation):
        return (
            evaluation["overall_score"] >= minimum_score
            and not evaluation["critical_issues"]
```

```
                and all(
                    score >= 3
                    for score in evaluation["scores"].values()
                )
            )

        # Accept a satisfactory first draft.
        if passes_quality_gate(initial_evaluation):
            return {
                "final_report": current_report,
                "initial_evaluation": initial_evaluation,
                "final_evaluation": initial_evaluation,
                "refined": False,
                "quality_gate_passed": True,
                "evaluation_history": evaluations,
            }

        # Otherwise, request one refinement.
        try:
            revised_report = optimize_research_report(
                ticker_symbol,
                research_question,
                specialist_evidence,
                current_report,
                initial_evaluation,
            )

            revised_evaluation = evaluate_research_report(
                ticker_symbol,
                research_question,
                specialist_evidence,
                revised_report,
            )

            evaluations.append(revised_evaluation)

            # Compare quality, penalizing critical issues.
            def quality_rank(evaluation):
                return (
                    evaluation["overall_score"]
                    - 1.5 * len(evaluation["critical_issues"])
                )

            if quality_rank(revised_evaluation) > quality_rank(
                initial_evaluation
            ):
                current_report = revised_report
                final_evaluation = revised_evaluation
            else:
                # Keep the original if the revision scores worse
                # or fails to improve the quality ranking.
                final_evaluation = initial_evaluation

        except Exception as exc:
            # Preserve the original draft if refinement fails.
            print(f"Optimization failed: {exc}")
            final_evaluation = initial_evaluation

        return {
            "final_report": current_report,
            "initial_evaluation": initial_evaluation,
            "final_evaluation": final_evaluation,
            "refined": current_report != initial_report,
            "quality_gate_passed": passes_quality_gate(
                final_evaluation
            ),
            "evaluation_history": evaluations,
        }
```

6. Agent Functions

6.1 Planning (`plan_research_steps`)

Given a stock ticker and research question, the planner inspects the available specialist descriptions in `AGENT_TOOL_REGISTRY` and generates a structured JSON array of ordered research steps.

6.2 Dynamic Tool Selection (`select_research_tools`)

Rather than blindly executing every tool, the router evaluates the user query, selects only the necessary specialist subset (`market`, `news`, `earnings`), verifies every selected tool against `AGENT_TOOL_REGISTRY`, and returns a transparent `routing_rationale`.

6.3 Self Reflection (`run_evaluator_optimizer`)

The system reflects on its own draft quality before returning an answer, enforcing a strict quality gate (mean score ≥ 4.0, zero critical issues, and every criterion score ≥ 3.0) and triggering automated refinement when needed.

6.4 Persistent Memory (`retrieve_historical_insights` & `save_enduring_insight`)

The system retains enduring analytical takeaways per ticker across runs and injects historical context into subsequent investigations.

```
def plan_research_steps(ticker_symbol: str, research_question: str) -> list[str]:
    tool_descriptions = {
        name: details['description']
        for name, details in AGENT_TOOL_REGISTRY.items()
    }
    result = _agent_json(f'''Create a short ordered research plan for this stock research request.
Return a JSON object with a 'steps' array of concise strings. Do not return code or factual claims.
Ticker: {ticker_symbol}
Research question: {research_question}
Available specialist tools: {json.dumps(tool_descriptions)}''')

    steps = result.get('steps')
    if not isinstance(steps, list) or not steps or any(not isinstance(step, str) or not step.strip() for step in steps):
        raise ValueError('The planner must return a non-empty list of non-empty research steps.')
    return [step.strip() for step in steps]
```

```
def select_research_tools(ticker_symbol: str, research_question: str) -> tuple[list[str], str]:
    tool_descriptions = {
        name: details['description']
        for name, details in AGENT_TOOL_REGISTRY.items()
    }
    result = _agent_json(f'''Select the specialist tools needed for this stock research request.
Return JSON with 'tools' as an ordered, non-empty array of tool names and 'rationale' as a brief reason.
Choose only from these registered names: {list(tool_descriptions)}.
For a broad company overview select all relevant specialists; for a focused question select only relevant tools.
Ticker: {ticker_symbol}
Research question: {research_question}
Registered tools: {json.dumps(tool_descriptions)}''')

    selected_tools = result.get('tools')
    rationale = result.get('rationale', '')
    if not isinstance(selected_tools, list) or not selected_tools:
        raise ValueError('The router must return a non-empty list of tool names.')
    if any(not isinstance(name, str) for name in selected_tools):
        raise ValueError('Every selected tool name must be a string.')
    invalid_tools = [name for name in selected_tools if name not in AGENT_TOOL_REGISTRY]
    if invalid_tools:
        raise ValueError(f'The router selected unregistered tools: {invalid_tools}')
    if not isinstance(rationale, str):
        raise ValueError('The routing rationale must be a string.')
    return list(dict.fromkeys(selected_tools)), rationale
```

7. Workflow 1: Prompt Chaining

Implemented in the News Agent (executed in Cell 49 above), where raw NewsAPI responses flow sequentially through five dependent stages: `ingest_news()` → `preprocess_news()` → `classify_news()` → `extract_information()` → `summarize_news()`. Each stage's output serves directly as the input to the next stage.

8. Workflow 2: Routing

Implemented in `select_research_tools()` and executed inside `run_investment_research()` below, where the supervisor classifies the user's intent and dynamically dispatches execution only to the relevant specialist agents registered in `AGENT_TOOL_REGISTRY`.

9. Workflow 3: Evaluator Optimizer

Implemented in `run_evaluator_optimizer()` and executed inside `run_investment_research()` below, establishing a closed loop `Generate Synthesis` → `Evaluate Quality` → `Optimize Draft` workflow that guarantees factual grounding and completeness.

```
def run_investment_research(ticker_symbol: str, research_question: str) -> dict:
```

```
    ticker_symbol = ticker_symbol.upper().strip()
    print("Ticker symbol: "+ticker_symbol+"\n")
    research_question = research_question.strip()
    if not ticker_symbol:
        raise ValueError('Ticker symbol cannot be empty.')
    if not research_question:
        raise ValueError('Research question cannot be empty.')

    research_plan = plan_research_steps(ticker_symbol, research_question)
    selected_tools, routing_rationale = select_research_tools(ticker_symbol, research_question)
    if not isinstance(selected_tools, list) or not selected_tools:
        raise ValueError('The router must return a non-empty list of tool names.')
    invalid_tools = [name for name in selected_tools if not isinstance(name, str) or name not in AGENT_TOOL_REGISTRY]
    if invalid_tools:
        raise ValueError(f'The router selected unregistered tools: {invalid_tools}')
    selected_tools = list(dict.fromkeys(selected_tools))
    if 'news' in selected_tools:
        ensure_news_api_key()

    specialist_evidence = {}
    for tool_name in selected_tools:
        specialist_evidence[tool_name] = AGENT_TOOL_REGISTRY[tool_name]['run'](ticker_symbol)

    evidence_json = json.dumps(specialist_evidence, indent=2, default=str)
    synthesis = _agent_text(f'''You are an investment research analyst. Answer the user's research question using only the
Do not invent figures or events. Clearly identify missing data, sources, and limitations. This is informational research, r
Ticker: {ticker_symbol}
Research question: {research_question}
Research plan: {json.dumps(research_plan)}
Specialist evidence: {evidence_json}''')

    # Existing synthesis has already been generated above.

    # Step 1: Evaluate the initial synthesis.
    # Step 2: Refine it if required.
    # Step 3: Re-evaluate and select the best draft.

    optimization_result = run_evaluator_optimizer(
        ticker_symbol=ticker_symbol,
        research_question=research_question,
        specialist_evidence=specialist_evidence,
        initial_report=synthesis,
    )

    # Preserve the existing result structure and add
    # the evaluator-optimizer results.
    return {
        "ticker": ticker_symbol,
        "research_question": research_question,
        "research_plan": research_plan,
        "selected_tools": selected_tools,
        "routing_rationale": routing_rationale,
        "specialist_evidence": specialist_evidence,

        # Best available report
        "synthesis": optimization_result["final_report"],

        # New fields
        "initial_evaluation": (
            optimization_result["initial_evaluation"]
        ),
        "final_evaluation": (
            optimization_result["final_evaluation"]
        ),
        "refined": optimization_result["refined"],
        "quality_gate_passed": (
            optimization_result["quality_gate_passed"]
        ),
        "evaluation_history": (
            optimization_result["evaluation_history"]
        ),
    }
```

10. End to End Demonstration

The execution below demonstrates the complete Investment Research Orchestrator on Microsoft Corporation (`MSFT`), displaying the generated research plan, dynamic specialist routing rationale, collected specialist evidence across market, news, and earnings

domains, and the synthesized research report.

```
# Example: change TICKER_SYMBOL or the research question to run a different investigation.
investment_research_result = run_investment_research(
    TICKER_SYMBOL,
    'Analyze recent earnings performance and year-over-year changes.',
)

print('Research plan:\n')

for step_number, step in enumerate(investment_research_result['research_plan'], start=1):
    print(f'{step_number}. {step}')

print('\nSelected specialists:', ', '.join(investment_research_result['selected_tools']))
print('Routing rationale:', investment_research_result['routing_rationale'])

print('\nSpecialist evidence:')
print(json.dumps(investment_research_result['specialist_evidence'], indent=2, default=str))

print('\nFinal synthesis:')
print(investment_research_result['synthesis'])
```

```
Ticker symbol: MSFT

Fetching real-world news for MSFT...
New Agent is classifying news...
New Agent is extracting facts...
New Agent is writing final summary...
Research plan:

1. Collect recent earnings data for MSFT from Yahoo Finance
2. Extract and analyze year-over-year (YoY) changes in revenue, EPS, and net income
3. Evaluate MSFT's recent earnings performance using Yahoo Finance metrics (e.g. returns, volatility, volume)
4. Gather and process news headlines from NewsAPI to identify any earnings-related news or sentiment
5. Compare and contrast MSFT's earnings performance with industry peers (using market data)

Selected specialists: market, news, earnings
Routing rationale: To provide a broad company overview, we need all relevant specialists. For the specific research questi

Specialist evidence:
{
  "market": "**Analysis of Microsoft Corporation (MSFT)**\n\n**1. Current Market Position**\n\n* Current price: $535.07\n*
  "news": {
    "preprocessed_input": "Publisher: Pypi.org | Headline: finzdata added to PyPI\nPublisher: Biztoc.com | Headline: S&P 5
    "classification_output": "**Story 1:**\n* Classification: Product Updates\n* Sentiment: Neutral\n\n**Story 2:**\n* Cla
    "extracted_facts_output": "Here are the extracted metrics, dates, and events for each story:\n\n**Story 1: finzdata ad
    "summary_output": "**1. Executive Overview**\n\nThe addition of finzdata to PyPI has expanded access to financial data
  },
  "earnings": "**1. Revenue**\n\n* Q2 2026: $90,007,000,000\n* Q1 2026: $82,886,000,000\n* Q4 2025: $81,273,000,000\n* Q3
}

Final synthesis:
**Earnings Analysis for MSFT**

Based on the provided data, here are the key observations and year-over-year changes in revenue, EPS, and net income for M

**Revenue:**

* Q2 2026: $90,007,000,000
* Q2 2025: $76,441,000,000
* Year-over-year change: 17.75%

**EPS:**

* Q2 2026: $4.81
* Q2 2025: $3.65
* Year-over-year change: 31.78%

**Net Income:**

* Q2 2026: $35,766,000,000
* Q2 2025: $27,233,000,000
* Year-over-year change: 31.33%

**Fiscal Year Revenue:**

* Annual 2026: $331,839,000,000
* Annual 2025: $281,724,000,000
* Year-over-year change: 17.79%
```

11. Evaluation Results

The output below displays the structured JSON scores from the Evaluator Agent (`factual_grounding: 5.0`, `completeness: 5.0`, `source_attribution: 5.0`, `financial_reasoning: 4.0`, `clarity: 5.0`, `overall_score: 4.8`), quality gate verification (`QUALITY GATE PASSED: True`), and the verified final report.

```
print("\nINITIAL EVALUATION")
print(
    json.dumps(
        investment_research_result["initial_evaluation"],
        indent=2,
    )
)

print("\nFINAL EVALUATION")
print(
    json.dumps(
        investment_research_result["final_evaluation"],
        indent=2,
    )
)

print("\nREFINED:", investment_research_result["refined"])

print(
    "QUALITY GATE PASSED:",
    investment_research_result["quality_gate_passed"],
)

print("\nFINAL RESEARCH REPORT\n")
print(investment_research_result["synthesis"])
```

```
INITIAL EVALUATION
{
  "scores": {
    "factual_grounding": 5.0,
    "completeness": 5.0,
    "source_attribution": 5.0,
    "financial_reasoning": 4.0,
    "clarity": 5.0
  },
  "overall_score": 4.8,
  "critical_issues": [],
  "issues": [
    "The draft report does not provide a detailed analysis of the earnings performance and year-over-year changes, and rel
  ],
  "revision_instructions": [
    "Provide a more in-depth analysis of the earnings performance and year-over-year changes, and identify potential areas
  ]
}

FINAL EVALUATION
{
  "scores": {
    "factual_grounding": 5.0,
    "completeness": 5.0,
    "source_attribution": 5.0,
    "financial_reasoning": 4.0,
    "clarity": 5.0
  },
  "overall_score": 4.8,
  "critical_issues": [],
  "issues": [
    "The draft report does not provide a detailed analysis of the earnings performance and year-over-year changes, and rel
  ],
  "revision_instructions": [
    "Provide a more in-depth analysis of the earnings performance and year-over-year changes, and identify potential areas
  ]
}

REFINED: False
QUALITY GATE PASSED: True

FINAL RESEARCH REPORT

**Earnings Analysis for MSFT**

Based on the provided data, here are the key observations and year-over-year changes in revenue, EPS, and net income for M

**Revenue:**

* Q2 2026: $90,007,000,000
* Q2 2025: $76,441,000,000
* Year-over-year change: 17.75%

**EPS:**
```

```
* Q2 2026: $4.81
```

12. Persistent Research Memory & Learning Across Runs (LangGraph)

This section integrates our orchestrator into a formal `LangGraph` workflow equipped with dual memory layers:

- **Short Term Thread Persistence (`SqliteSaver`):** Checkpoints the `ResearchState` (`ticker`, `question`, `final_report`) to a local SQLite database (`research_memory.db`).
- **Long Term Cross Run Memory (`InMemoryStore`):** Organizes enduring analytical insights by namespace `("memory", ticker)`. After Run 1 finishes, `save_enduring_insight()` extracts two structural takeaways and writes them to the store. When Run 2 begins, `retrieve_historical_insights()` recalls those notes and injects them into the research context.

> **Important Architectural Note:** In alignment with agentic AI design principles, "learning across runs" refers to **persistent research memory and accumulated analytical notes** stored in an external state database and injected via context augmentation; it does not retrain or modify the underlying `llama3.1:8b` model weights.

```
import sqlite3
import uuid
import json
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.sqlite import SqliteSaver

# ------------------------------------------------------------
# 1. LONG-TERM MEMORY HELPERS
# ------------------------------------------------------------
def retrieve_historical_insights(ticker: str, store: InMemoryStore) -> str:
    """Reads past persistent memories for a specific ticker."""
    namespace = ("memory", ticker)
    past_memories = store.search(namespace, limit=3)

    if not past_memories:
        return "No historical notes available."

    return "\n".join([mem.value["insights"] for mem in past_memories])

def save_enduring_insight(ticker: str, final_report: str, store: InMemoryStore):
    """Writes a new enduring insight to the Store after a run."""
    prompt = f"""
    Extract exactly two brief, enduring analytical takeaways from this report on {ticker}.
    Focus on structural market changes, long-term risks, or management goals.
    Do not include daily price movements. Output ONLY the concise takeaways.

    Report: {final_report}
    """
    # Using your existing _agent_text helper
    insights = _agent_text(prompt).strip()

    namespace = ("memory", ticker)
    memory_id = str(uuid.uuid4())
    store.put(namespace, memory_id, {"insights": insights})
    print(f"\n[Memory Saved] 🧠 New insights permanently stored for {ticker}.")

# ------------------------------------------------------------
# 2. LANGGRAPH STATE AND NODE
# ------------------------------------------------------------
class ResearchState(TypedDict):
    ticker: str
    question: str
    final_report: str

# Accept state directly; store is accessible in scope
def research_node(state: ResearchState):
    ticker = state["ticker"]
    question = state["question"]

    print(f"\n--- STARTING LANGGRAPH RUN FOR {ticker} ---")

    # 1. READ MEMORY: Check if we know anything about this company already
    historical_notes = retrieve_historical_insights(ticker, store)
    print(f"\n[Recalled Memory]:\n{historical_notes}\n")

    # Inject the memory directly into the user's question so the specialists see it
    contextualized_question = f"""
    {question}
```

```
    Keep these historical notes in mind while answering:
    {historical_notes}
    """

    # 2. EXECUTE EXISTING ORCHESTRATOR
    result = run_investment_research(ticker, contextualized_question)
    final_report = result["synthesis"]

    # 3. WRITE MEMORY: Save new learnings for future runs
    save_enduring_insight(ticker, final_report, store)

    return {"final_report": final_report}

# ------------------------------------------------------------
# 3. BUILD AND COMPILE THE GRAPH
# ------------------------------------------------------------
builder = StateGraph(ResearchState)

builder.add_node("researcher", research_node)
builder.add_edge(START, "researcher")
builder.add_edge("researcher", END)

# Initialize memory databases
store = InMemoryStore()
conn = sqlite3.connect("research_memory.db", check_same_thread=False)
checkpointer = SqliteSaver(conn)

# Compile graph
app = builder.compile(
    checkpointer=checkpointer,
    store=store
)
```

Execution & Memory Validation

Run this cell to test the agent. The first run will generate a foundational report and save insights to the `InMemoryStore`. The second run will immediately recall those insights to inform the next analysis.

```
# Create a unique thread ID for this session
config = {"configurable": {"thread_id": "msft_session_001"}}

print("=== RUN 1: INITIAL RESEARCH ===")
initial_state = {
    "ticker": "MSFT",
    "question": "Analyze recent earnings performance and year-over-year changes."
}
# The app will run, generate the report, and internally save insights to the Store
run_1_result = app.invoke(initial_state, config=config)


print("\n\n=== RUN 2: PROVING LONG-TERM MEMORY ===")
follow_up_state = {
    "ticker": "MSFT",
    "question": "What are the immediate market implications for the company right now?"
}
# Notice how the output console will print the [Recalled Memory] extracted from Run 1!
run_2_result = app.invoke(follow_up_state, config=config)

print("\n=== FINAL OUTPUT FROM RUN 2 ===")
print(run_2_result["final_report"])
```

```
=== RUN 1: INITIAL RESEARCH ===

--- STARTING LANGGRAPH RUN FOR MSFT ---

[Recalled Memory]:
No historical notes available.

Ticker symbol: MSFT

Fetching real-world news for MSFT...
New Agent is classifying news...
New Agent is extracting facts...
New Agent is writing final summary...

[Memory Saved] 🧠 New insights permanently stored for MSFT.


=== RUN 2: PROVING LONG-TERM MEMORY ===

--- STARTING LANGGRAPH RUN FOR MSFT ---
```

```
[Recalled Memory]:
Here are two brief, enduring analytical takeaways from the report on MSFT:

1. **Rising Valuation Concerns**: Microsoft's PE ratio is above the industry average, indicating potential overvaluation a
2. **Increasing Regulatory Risks**: The report highlights regulatory actions in the US and Italy affecting Microsoft and t

Ticker symbol: MSFT

Fetching real-world news for MSFT...
New Agent is classifying news...
New Agent is extracting facts...
New Agent is writing final summary...

[Memory Saved] 🧠 New insights permanently stored for MSFT.

=== FINAL OUTPUT FROM RUN 2 ===
Based on the provided specialist evidence, here's the analysis of the immediate market implications for Microsoft Corporat

**Market Trends and Performance**

* The stock has experienced a strong upward trend in its price over the past three months, with a 7.96% return in the last
* The current market position indicates a moderate increase in price, with a daily change of $12.46 (2.38% increase) and a
* The annualized volatility is high, indicating a higher risk associated with the stock.

**Valuation Metrics and Risks**

* The PE ratio is high, at 29.11, which may indicate a potentially overvalued stock.
* The forward PE ratio is lower than the PE ratio, at 22.60, which may suggest a decline in growth expectations.
* The high PE ratio and forward PE ratio may indicate a potentially overvalued stock.

**Regulatory Risks**

* Recent news headlines indicate that Microsoft's stock is in focus due to US targeting green card filings for H-1B worker

**Earnings Performance**
```

13. Limitations

- **Data Source Granularity:** Yahoo Finance income statements provide historical actuals but do not include forward management guidance or earnings call transcripts. Integrating SEC EDGAR 10 K and 10 Q filings via RAG would enrich forward looking analysis.
- **News Ingestion Scope:** The News Agent currently processes the top 5 recent headlines per run to maintain reasonable local inference speed.
- **Sequential Local Inference Latency:** Running the multi stage prompt chain, specialist agents, synthesis, evaluation, and memory extraction sequentially on a local `llama3.1:8b` instance requires 10 to 12 LLM calls per run.

14. Conclusion

This project demonstrates an autonomous Multi Agent Financial Analysis System built with Python, Ollama, Yahoo Finance, NewsAPI, and LangGraph. By combining step by step research planning, selective tool routing, a 5 stage news prompt chain, quantitative self reflection via an Evaluator Optimizer quality gate, and cross run persistent memory, the architecture produces grounded, auditable equity research.

15. GitHub Repository

- **Deliverables & Version Control:** Maintained in our team GitHub repository : https://github.com/kunalgurtoo6-dev/AAI520_MultiAgent_Finance.

Double-click (or enter) to edit